



```
In [10... import numpy as np
import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)
pd.set_option("display.max_rows", 100)

FILE_NAME = "/content/sample_data/rahul_transactions.csv"
```

```
In [11... raw = pd.read_csv(FILE_NAME)
raw_rows = len(raw)
raw_columns = list(raw.columns)

df = raw.copy()
df.columns = [str(c).strip() for c in df.columns]

df["date"] = pd.to_datetime(
    df["Date"].astype(str).str.strip(),
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

df["amount"] = (
    df["Amount"].astype(str)
    .str.replace("₹", "", regex=False)
    .str.replace("Rs.", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.strip()
)
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")

df["type"] = (
    df["Type"].astype(str).str.strip().str.lower()
    .replace({"dr": "debit", "debit": "debit", "cr": "credit", "credit": "credit"})
)

df["Mode"] = df["Mode"].replace(r"^\s*$", np.nan, regex=True)

duplicate_rows = int(df.duplicated().sum())
bad_dates = int(df["date"].isna().sum())
bad_amounts = int(df["amount"].isna().sum())

df = df.drop_duplicates().dropna(subset=["date", "amount", "type"]).copy()
df = df.sort_values(["date", "Time", "amount"], kind="stable").reset_index(drop=True)

df["hour"] = pd.to_numeric(df["Time"].astype(str).str[:2], errors="coerce")
df["month"] = df["date"].dt.month
df["month_name"] = df["date"].dt.strftime("%b")
df["day_of_week"] = df["date"].dt.day_name()

print(f"Raw rows: {raw_rows}")
print(f"Exact duplicate rows removed: {duplicate_rows}")
```

```

print(f"Rows after cleaning: {len(df)}")
print(f>Date range: {df['date'].min().date()} to {df['date'].max().date()}")
print(f"Unparseable dates: {bad_dates}; unparseable amounts: {bad_amounts}")
print(f"\nStandardised type counts:")
print(df["type"].value_counts())

```

Raw rows: 1328
 Exact duplicate rows removed: 18
 Rows after cleaning: 1310
 Date range: 2024-01-01 to 2024-06-30
 Unparseable dates: 0; unparseable amounts: 0

Standardised type counts:
 type
 debit 1304
 credit 6
 Name: count, dtype: int64

```

In [15... vendor_keywords = {
    "Swiggy Instamart": ["SWIGGY-INSTAMART", "INSTAMART", "BUNDL TECH-INS
    "Swiggy": ["SWIGGY", "BUNDL"],
    "Zomato": ["ZOMATO"],
    "Amazon": ["AMAZON", "AMZN"],
    "Zepto": ["ZEPTO"],
    "Blinkit": ["BLINKIT", "GROFERS"],
    "Myntra": ["MYNTRA"],
    "Flipkart": ["FLIPKART", "FKART"],
    "Nykaa": ["NYKAA", "FSN E-COMMERCE"],
    "Uber": ["UBER", "ANI TECHNOLOGIES"],
    "Ola": ["OLA CABS", "OLA ELECTRIC", "POS OLA", "OLA-PRIME"],
    "Rapido": ["RAPIDO", "ROPEN TRANSPORTATION"],
    "BMTc": ["BMTc", "TUMMOC"],
    "Zerodha": ["ZERODHA"],
    "Groww": ["GROWw"],
    "BookMyShow": ["BMS MOVIE", "BOOKMYSHOW", "BIGTREE ENTERTAINMENT"],
    "Netflix": ["NETFLIX"],
    "Spotify": ["SPOTIFY"],
    "YouTube": ["YOUTUBE"],
    "Prime Video": ["PRIME VIDEO", "AMZN PRIME", "AMAZON-PRIME"],
    "Hotstar": ["HOTSTAR", "DISNEY HOTSTAR", "STAR INDIA"],
    "Jio": ["JIO"],
    "Airtel": ["AIRTEL", "BHARTI AIRTEL"],
    "VI": ["VODAFONE IDEA", "VI POSTPAID", "VI-RECHARGE"],
    "BESCOM": ["BESCOM", "BANGALORE ELEC SUPPLY"],
    "BWSSB": ["BWSSB"],
    "BigBasket": ["BIGBASKET"],
    "DMart": ["DMART", "AVENUE SUPERMARTS"],
    "Cafe Coffee Day": ["CAFE COFFEE DAY", "COFFEE DAY GLOBAL", "UPI-CCD"],
    "Third Wave Coffee": ["THIRDWAVE", "THIRD WAVE", "TWC INDIA"],
    "Starbucks": ["STARBUCKS"],
    "McDonald's": ["MCDONALD"],
    "Domino's": ["DOMINO"],
    "Empire Restaurant": ["EMPIRE RESTAURANT"],
    "Meghana Foods": ["MEGHANA FOODS"],
    "Truffles": ["TRUFFLES"],

```

```

    "Dineout": ["DINEOUT"],
    "IndianOil": ["INDIAN OIL", "INDIANOIL", "IOC"],
    "HP Fuel": ["HP PETROL", "HPCL"],
    "BPCL": ["BPCL"],
    "Paytm": ["PAYTM"],
    "PhonePe": ["PHONEPE"],
    "Amazon Pay": ["AMAZONPAY"],
    "Rent/Landlord": ["RENT-LANDLORD"],
}

def extract_vendor(description):
    text = str(description).upper().strip()
    if text.startswith("ATM-WDL") or "ATM WITHDRAW" in text:
        return "Cash Withdrawal"
    for vendor, keywords in vendor_keywords.items():
        if any(keyword in text for keyword in keywords):
            return vendor
    if text.startswith("UPI-") and "@" in text:
        return "P2P Transfer"
    if "RESTAURANT-" in text or text == "UPI-RESTAURANT":
        return "Restaurant / P2P"
    return "Uncategorised"

df["vendor_clean"] = df["Description"].apply(extract_vendor)

print("Top canonical vendors by transaction count:")
print(df["vendor_clean"].value_counts().head(20))

```

Top canonical vendors by transaction count:

```

vendor_clean
Swiggy          176
Zomato          121
Uber            89
Amazon          86
Swiggy Instamart 67
Ola             59
Zepto           58
Rapido          55
Blinkit         55
Flipkart        47
Starbucks       42
Uncategorised   39
BMTc            37
Third Wave Coffee 31
P2P Transfer    28
Cafe Coffee Day 26
DMart           22
Myntra          20
Nykaa           19
Paytm           17
Name: count, dtype: int64

```

```

In [16... uncategorised = (
    df.loc[df["vendor_clean"].eq("Uncategorised"), "Description"]
    .value_counts()

```

```
)
print(f"Uncategorised transaction rows: {int((df['vendor_clean'] == 'Uncat
print(uncategorised)
```

Uncategorised transaction rows: 39

```
Description
KIRANAKART TECH          13
POS BANGALORE RESTAURANT 12
INNOVATIVE RETAIL        8
NEFT-TECHCRUSH LABS-SALARY MAY24 6
Name: count, dtype: int64
```

```
In [17... vendor_to_category = {
    "Swiggy": "Food Delivery", "Zomato": "Food Delivery",
    "Swiggy Instamart": "Quick Commerce", "Zepto": "Quick Commerce", "Bli
    "BigBasket": "Groceries", "DMart": "Groceries",
    "Amazon": "E-commerce", "Amazon Pay": "E-commerce", "Myntra": "E-comm
    "Uber": "Transport", "Ola": "Transport", "Rapido": "Transport", "BMT
    "Cafe Coffee Day": "Cafe", "Third Wave Coffee": "Cafe", "Starbucks":
    "McDonald's": "Restaurants", "Domino's": "Restaurants", "Empire Resta
    "Meghana Foods": "Restaurants", "Truffles": "Restaurants", "Dineout":
    "Netflix": "Subscriptions", "Spotify": "Subscriptions", "YouTube": "S
    "Prime Video": "Subscriptions", "Hotstar": "Subscriptions",
    "Jio": "Utilities", "Airtel": "Utilities", "VI": "Utilities", "BESCOM
    "Zerodha": "Investments", "Groww": "Investments",
    "BookMyShow": "Entertainment",
    "IndianOil": "Fuel", "HP Fuel": "Fuel", "BPCL": "Fuel",
    "P2P Transfer": "Personal Transfer", "Rent/Landlord": "Housing", "Cas
    "Paytm": "Payments", "PhonePe": "Payments",
}

df["category"] = df["vendor_clean"].map(vendor_to_category).fillna("Uncat
print(df["category"].value_counts())
```

```
category
Food Delivery      297
Transport          240
Quick Commerce     180
E-commerce         172
Cafe               99
Restaurants        44
Utilities          43
Uncategorised      39
Groceries          33
Subscriptions      31
Fuel              28
Personal Transfer  28
Investments        23
Payments           17
Cash Withdrawal    17
Entertainment      13
Housing            6
Name: count, dtype: int64
```

```
In [18... debits = df[df["type"].eq("debit")].copy()
```

```

credits = df[df["type"].eq("credit")].copy()

total_debits = float(debits["amount"].sum())
total_credits = float(credits["amount"].sum())
net_change = total_credits - total_debits
savings_rate = net_change / total_credits * 100 if total_credits else np.

consumption = debits[~debits["category"].isin(["Personal Transfer", "Cash
consumption_total = float(consumption["amount"].sum())

category_summary = (
    consumption.groupby("category")["amount"]
    .agg(total="sum", transactions="count")
    .sort_values("total", ascending=False)
)
category_summary["pct_of_consumption"] = category_summary["total"] / cons

vendor_summary = (
    consumption.groupby("vendor_clean")["amount"]
    .agg(total="sum", transactions="count")
    .sort_values("total", ascending=False)
)

print("=" * 78)
print("SPENDDNA - EXECUTIVE SUMMARY")
print("=" * 78)
print(f"Raw transactions      : {raw_rows:,}")
print(f"Clean transactions     : {len(df):,}")
print(f"Credits                  : ₹{total_credits:,.0f}")
print(f"Debits                   : ₹{total_debits:,.0f}")
print(f"Net cash-flow change    : ₹{net_change:,.0f}")
print(f"Savings rate vs credit: {savings_rate:.1f}%")
print(f"Consumption spend       : ₹{consumption_total:,.0f}")
print("\nTOP CATEGORIES")
print(category_summary.round(1).head(10))
print("\nTOP VENDORS")
print(vendor_summary.head(15))

```

SPENDDNA – EXECUTIVE SUMMARY

Raw transactions : 1,328
 Clean transactions : 1,310
 Credits : ₹509,774
 Debits : ₹1,678,901
 Net cash-flow change : ₹-1,169,127
 Savings rate vs credit: -229.3%
 Consumption spend : ₹1,606,035

TOP CATEGORIES

category	total	transactions	pct_of_consumption
E-commerce	603877.0	172	37.6
Investments	248160.0	23	15.5
Food Delivery	129054.0	297	8.0
Housing	108000.0	6	6.7
Quick Commerce	90765.0	180	5.7
Fuel	89303.0	28	5.6
Restaurants	72759.0	44	4.5
Transport	54707.0	240	3.4
Groceries	50208.0	33	3.1
Utilities	41914.0	43	2.6

TOP VENDORS

vendor_clean	total	transactions
Amazon	328530.0	86
Zerodha	210000.0	14
Flipkart	177510.0	47
Rent/Landlord	108000.0	6
Swiggy	73738.0	176
Myntra	69529.0	20
IndianOil	56335.0	17
Zomato	55316.0	121
Uncategorised	41629.0	33
Groww	38160.0	9
DMart	37463.0	22
Blinkit	31995.0	55
Swiggy Instamart	30613.0	67
Uber	29384.0	89
Nykaa	28308.0	19

```
In [19... monthly_category = consumption.pivot_table(
    values="amount", index="category", columns="month_name", aggfunc="sum
)
month_order = [m for m in ["Jan", "Feb", "Mar", "Apr", "May", "Jun"] if m
monthly_category = monthly_category.reindex(columns=month_order)
print("MONTHLY SPEND BY CATEGORY")
print(monthly_category.round(0))

monthly_total = consumption.groupby("month_name")["amount"].sum().reindex
print("\nTOTAL CONSUMPTION BY MONTH")
```

```
print(monthly_total.round(0))

first_month = month_order[0]
last_month = month_order[-1]
trend = pd.DataFrame({"first_month": monthly_category[first_month], "last
trend["pct_change"] = np.where(trend["first_month"] != 0, (trend["last_mo
print("\nFIRST-TO-LAST MONTH CATEGORY CHANGE")
print(trend.sort_values("pct_change", ascending=False).round(1))
```

MONTHLY SPEND BY CATEGORY

month_name	Jan	Feb	Mar	Apr	May	Jun
category						
Cafe	3690.0	4273.0	5448.0	6564.0	5668.0	5802.0
E-commerce	98623.0	94011.0	108215.0	69219.0	95776.0	138033.0
Entertainment	1263.0	474.0	2418.0	2224.0	0.0	1914.0
Food Delivery	20890.0	21452.0	20850.0	23054.0	22167.0	20641.0
Fuel	30322.0	2079.0	26164.0	18718.0	9138.0	2882.0
Groceries	17649.0	7517.0	5523.0	6225.0	7862.0	5432.0
Housing	18000.0	18000.0	18000.0	18000.0	18000.0	18000.0
Investments	38432.0	15000.0	68644.0	54126.0	48628.0	23330.0
Payments	0.0	2895.0	3590.0	2538.0	2541.0	5886.0
Quick Commerce	12797.0	17465.0	17297.0	15259.0	14570.0	13377.0
Restaurants	7376.0	9067.0	23864.0	5173.0	16146.0	11133.0
Subscriptions	2767.0	4628.0	3509.0	2168.0	1844.0	3555.0
Transport	10551.0	9111.0	6857.0	9284.0	12322.0	6582.0
Uncategorised	8944.0	10864.0	2307.0	6836.0	6073.0	6605.0
Utilities	9157.0	6870.0	5497.0	5831.0	4340.0	10219.0

TOTAL CONSUMPTION BY MONTH

month_name	
Jan	280461.0
Feb	223706.0
Mar	318183.0
Apr	245219.0
May	265075.0
Jun	273391.0

Name: amount, dtype: float64

FIRST-TO-LAST MONTH CATEGORY CHANGE

	first_month	last_month	pct_change
category			
Cafe	3690.0	5802.0	57.2
Entertainment	1263.0	1914.0	51.5
Restaurants	7376.0	11133.0	50.9
E-commerce	98623.0	138033.0	40.0
Subscriptions	2767.0	3555.0	28.5
Utilities	9157.0	10219.0	11.6
Quick Commerce	12797.0	13377.0	4.5
Housing	18000.0	18000.0	0.0
Food Delivery	20890.0	20641.0	-1.2
Uncategorised	8944.0	6605.0	-26.2
Transport	10551.0	6582.0	-37.6
Investments	38432.0	23330.0	-39.3
Groceries	17649.0	5432.0	-69.2
Fuel	30322.0	2882.0	-90.5
Payments	0.0	5886.0	NaN

```
In [20... time_matrix = consumption.pivot_table(
        values="amount", index="category", columns="hour", aggfunc="sum", fil
    )
    print("SPEND BY CATEGORY AND HOUR")
    print(time_matrix.round(0))
```

```

peak_hour = consumption.groupby("hour")["amount"].sum().idxmax()
peak_hour_amount = consumption.groupby("hour")["amount"].sum().max()
print(f"\nOverall peak spending hour: {int(peak_hour):02d}:00 (₹{peak_hou

food_delivery = debits[debits["category"].eq("Food Delivery")].copy()
late_food = food_delivery[food_delivery["hour"].isin([21,22,23,0,1,2])]
late_food_pct = len(late_food) / len(food_delivery) * 100 if len(food_del
print(f"Food Delivery transactions: {len(food_delivery)}")
print(f"Late-night Food Delivery share (21:00–02:59): {late_food_pct:.1f}

print("\nPEAK HOUR BY CATEGORY")
for category in time_matrix.index:
    h = time_matrix.loc[category].idxmax()
    print(f"{category:<22} {int(h):02d}:00")

```

SPEND BY CATEGORY AND HOUR

hour 15 \ category	0	1	2	3	4	5	6	7
Cafe	872.0	549.0	0.0	316.0	0.0	0.0	179.0	480.0
E-commerce	18650.0	9801.0	10246.0	14865.0	12018.0	17258.0	9249.0	10916.0
Entertainment	562.0	620.0	0.0	0.0	0.0	0.0	0.0	311.0
Food Delivery	438.0	2598.0	928.0	1702.0	2133.0	3488.0	238.0	799.0
Fuel	2675.0	2048.0	0.0	676.0	2010.0	2105.0	0.0	0.0
Groceries	3895.0	1536.0	2352.0	7080.0	1169.0	0.0	5265.0	0.0
Housing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Investments	4883.0	15000.0	0.0	0.0	18834.0	4496.0	37717.0	0.0
Payments	0.0	0.0	1336.0	0.0	0.0	0.0	0.0	0.0
Quick Commerce	1936.0	1821.0	1143.0	1165.0	1253.0	1448.0	787.0	951.0
Restaurants	1411.0	960.0	0.0	1714.0	0.0	496.0	0.0	0.0
Subscriptions	0.0	1038.0	1295.0	4029.0	4685.0	745.0	657.0	1157.0
Transport	491.0	260.0	358.0	337.0	323.0	2005.0	555.0	3924.0
Uncategorised	0.0	0.0	0.0	2984.0	0.0	1630.0	859.0	0.0
Utilities	1941.0	2651.0	2625.0	0.0	1222.0	0.0	0.0	1400.0

hour category	16	17	18	19	20	21	22	23
Cafe	3079.0	3552.0	2562.0	584.0	1491.0	0.0	0.0	764.0
E-commerce	52046.0	35378.0	35272.0	12668.0	51190.0	35095.0	16182.0	15403.0
Entertainment	1881.0	0.0	1270.0	417.0	1426.0	0.0	343.0	0.0
Food Delivery	4622.0	3667.0	12698.0	15533.0	14973.0	9083.0	8201.0	4437.0
Fuel	9343.0	0.0	1950.0	8351.0	705.0	11845.0	0.0	1641.0
Groceries	0.0	0.0	910.0	1248.0	2566.0	3688.0	0.0	2536.0
Housing	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Investments	0.0	0.0	15000.0	4476.0	15000.0	0.0	15000.0	18628.0
Payments	0.0	1346.0	2984.0	1610.0	1789.0	1202.0	1145.0	0.0
Quick Commerce	4606.0	1232.0	5944.0	8759.0	10257.0	9291.0	4143.0	1894.0
Restaurants	10922.0	2748.0	6774.0	7005.0	14020.0	3380.0	1637.0	1256.0
Subscriptions	0.0	0.0	0.0	1435.0	0.0	119.0	0.0	0.0
Transport	3325.0	3159.0	3203.0	3550.0	4647.0	1622.0	1852.0	1458.0
Uncategorised	2810.0	0.0	1199.0	0.0	12315.0	2216.0	344.0	0.0
Utilities	602.0	1344.0	5468.0	708.0	4162.0	0.0	835.0	386.0

Overall peak spending hour: 20:00 (₹134,541)

Food Delivery transactions: 297

Late-night Food Delivery share (21:00–02:59): 21.2%

PEAK HOUR BY CATEGORY

Cafe	10:00
E-commerce	14:00
Entertainment	16:00
Food Delivery	19:00
Fuel	15:00
Groceries	03:00
Housing	13:00
Investments	10:00
Payments	18:00
Quick Commerce	20:00
Restaurants	20:00
Subscriptions	04:00
Transport	20:00
Uncategorised	20:00
Utilities	18:00

```
In [21... category_mean = df.groupby("category")["amount"].transform("mean")
category_std = df.groupby("category")["amount"].transform("std")
df["z_score"] = np.where(category_std > 0, (df["amount"] - category_mean)

anomalies = df[(df["type"].eq("debit")) & (df["z_score"] > 2)].sort_value
print(f"Anomalies detected: {len(anomalies)}")
print(anomalies[["date", "Description", "vendor_clean", "category", "amount",
```

Anomalies detected: 20

	date	Description	vendor_clean	category	amount	z_score
349	2024-06-26	AMAZONIN MARKETPLACE	Amazon	E-commerce	22008.0	4.090
484	2024-02-07	UPI-AMAZONPAY@HDFCBANK	Amazon	E-commerce	21986.0	4.085
947	2024-06-22	POS DINEOUT	Dineout	Restaurants	7935.0	3.637
631	2024-03-31	POS MEGHANA FOODS	Meghana Foods	Restaurants	7931.0	3.635
956	2024-03-05	UPI-AMAZONPAY@HDFCBANK	Amazon	E-commerce	19917.0	3.627
841	2024-03-04	POS TRUFFLES	Truffles	Restaurants	7441.0	3.351
410	2024-03-02	AMAZON IN	Amazon	E-commerce	18273.0	3.264
669	2024-05-27	UPI-FLIPKART@HDFCBANK	Flipkart	E-commerce	17831.0	3.166
358	2024-06-25	Amazon Pay India	Amazon	E-commerce	17504.0	3.094
370	2024-06-03	UPI-FLIPKART@HDFCBANK	Flipkart	E-commerce	16369.0	2.843

```
In [ ]: debit_total = total_debits

def pct_of_debits(categories):
    return debits[debits["category"].isin(categories)][["amount"].sum() /
```

```

food_pct = pct_of_debits(["Food Delivery", "Restaurants", "Cafe"])
quick_pct = pct_of_debits(["Quick Commerce"])
ecom_pct = pct_of_debits(["E-commerce"])
investment_pct = pct_of_debits(["Investments"])
transport_pct = pct_of_debits(["Transport"])
subscription_vendors = df.loc[df["category"].eq("Subscriptions"), "vendor"]

archetypes = []
if food_pct > 25: archetypes.append(("THE FOODIE", f"Food-related debits"))
if quick_pct > 15: archetypes.append(("THE QUICK COMMERCE JUNKIE", f"Quick Commerce"))
if ecom_pct > 15: archetypes.append(("THE SHOPAHOLIC", f"E-commerce = {ecom_pct}"))
if investment_pct > 15: archetypes.append(("THE INVESTOR", f"Investments = {investment_pct}"))
if late_food_pct > 50: archetypes.append(("THE LATE-NIGHT SNACKER", f"Late-night food = {late_food_pct}"))
if transport_pct > 10: archetypes.append(("THE CAB COMMUTER", f"Transport = {transport_pct}"))
if subscription_vendors >= 5: archetypes.append(("THE SUBSCRIPTION LOVER", f"Subscription vendors = {subscription_vendors}"))
if savings_rate < 10: archetypes.append(("THE YOLO SPENDER", f"Savings rate = {savings_rate}"))
if savings_rate > 40: archetypes.append(("THE DISCIPLINED SAVER", f"Savings rate = {savings_rate}"))

print("RAHUL'S SPENDING ARCHETYPES")
for name, evidence in archetypes:
    print(f"→ {name:<30} {evidence}")

```

In [22...]

```

day_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
day_spend = consumption.groupby("day_of_week")["amount"].sum().reindex(day_order)
print(day_spend.round(0))
weekend = day_spend.loc[["Saturday", "Sunday"]].sum()
weekday = day_spend.loc[["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]].sum()
print(f"\nWeekend spend: ₹{weekend:,.0f}")
print(f"Weekday spend: ₹{weekday:,.0f}")
print(f"Weekend as % of weekday spend: {weekend/weekday*100:.1f}%")

```

```

day_of_week
Monday      242327.0
Tuesday     245368.0
Wednesday   292440.0
Thursday    183076.0
Friday       182080.0
Saturday    247028.0
Sunday       213716.0
Name: amount, dtype: float64

```

```

Weekend spend: ₹460,744
Weekday spend: ₹1,145,291
Weekend as % of weekday spend: 40.2%

```